C++프로그래밍 및 실습 — 프로젝트 실습 보고서

간단한 MUD 게임 구현 (week10)

김진성 (213549)

1. 서론

1.1 프로젝트 목적 및 배경

7주차까지 배운 2차원 배열, 사용자 정의 함수, 조건문·반복문 등을 실제 프로그램에 적용해 보는 실습으로 진행하였다. 기본 골격이 되는 MUD 게임에 체력(HP) 시스템과 칸 상태 처리 기능을 직접 추가·수정하는 것을 중심으로 작업하였다. 그래픽 없이 텍스트로만 작동하는 게임이라 조건과 출력을 잘못 설계하면 화면에 바로 드러나기 때문에, 실행하며 고쳐 쓰는 연습에도 좋은 과제였다.

1.2 목표

- 5×5 2차원 배열 지도에서 상/하/좌/우로 이동해 목적지에 도착하는 MUD 게임 완성
- HP 20 시작, 이동 시 1 감소, 적 -2 / 포션 +2 등 개선 요구사항 반영
- 칸 상태 처리를 checkState()라는 사용자 정의 함수로 분리

2. 요구사항

2.1 사용자 요구사항

1. "상", "하", "좌", "우", "지도", "종료" 중 하나를 입력받는다. 그 외 입력에는 에러 메시지 후 재입력받는다.
2. 지도 밖으로 나가려 하면 에러 메시지를 출력하고 제자리로 돌아간다.
3. "지도" 입력 시 전체 지도와 현재 위치를 출력한다.
4. 목적지에 도착하면 "성공"을 출력하고 종료한다.
5. 유저는 HP 20으로 시작하고, **이동할 때마다 HP가 1씩 감소**한다. 명령을 입력받을 때마다 HP를 함께 출력하며, HP가 0이 되면 "실패"를 출력하고 종료한다.
6. 아이템/포션/적을 만나면 "{X}이/가 있습니다" 형식의 메시지를 출력한다. 적을 만나면 HP 2 감소, 포션을 만나면 HP 2 증가하며, 물건은 사라지지 않는다.

2.2 기능 계획

1. 무한 반복으로 명령을 입력받고 해당 기능 호출
2. 이동 처리: 좌표 증감 → 경계 검사 → 통과 시 지도 출력 → 칸 상태 처리 → 이동 비용으로 HP 1 감소

3. 종료 조건 검사: 목적지 도착(성공) / HP 0 이하(실패)

2.3 함수 계획

main()	게임 전체 흐름: 명령 입력 → 이동 → 상태 처리 및 HP 차감 → 종료 조건 검사
displayMap(map, user_x, user_y)	전체 지도와 사용자 위치 출력
checkXY(user_x, mapX, user_y, mapY)	이동 좌표가 지도 내부인지 검사 → bool
checkGoal(map, user_x, user_y)	현재 좌표가 목적지(4)인지 검사 → bool
checkState(map, user_x, user_y, int& hp)	도착한 칸의 상태에 따라 메시지 출력과 HP 증감. hp는 참조로 받아 실제 체력을 갱신

3. 설계 및 구현

3.1 메인 함수 — 초기화 및 명령 입력



그림 1. 지도·HP·유저 위치 초기화

입력	map[mapY][mapX] = 5×5 지도(0 빈칸, 1 아이템, 2 적, 3 포션, 4 목적지) / hp = 20 / user_x, user_y = 시작 위치 (0,0)
반환값	없음 (게임 종료 시 0 반환)
결과	게임을 시작할 수 있는 초기 상태
설명	지도 값 하나가 곧 칸의 종류를 뜻하는 상태코드가 되도록 선언·초기화했다. 체력 hp = 20으로 시작하는 항목도 이 부분에 추가했다.

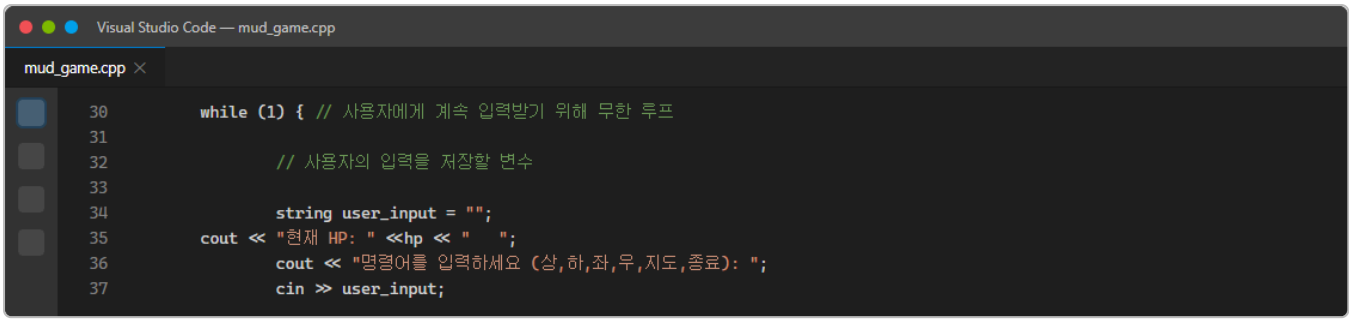


그림 2. 명령 입력 루프

`while(1)` 루프가 돌 때마다 HP를 먼저 출력한 뒤 명령을 입력받는다. 여섯 명령 외의 입력은 `else`에서 "잘못된 입력입니다"를 출력하고 `continue`로 재입력을 받는다.

3.2 메인 함수 — 이동 처리 ('상' 명령)

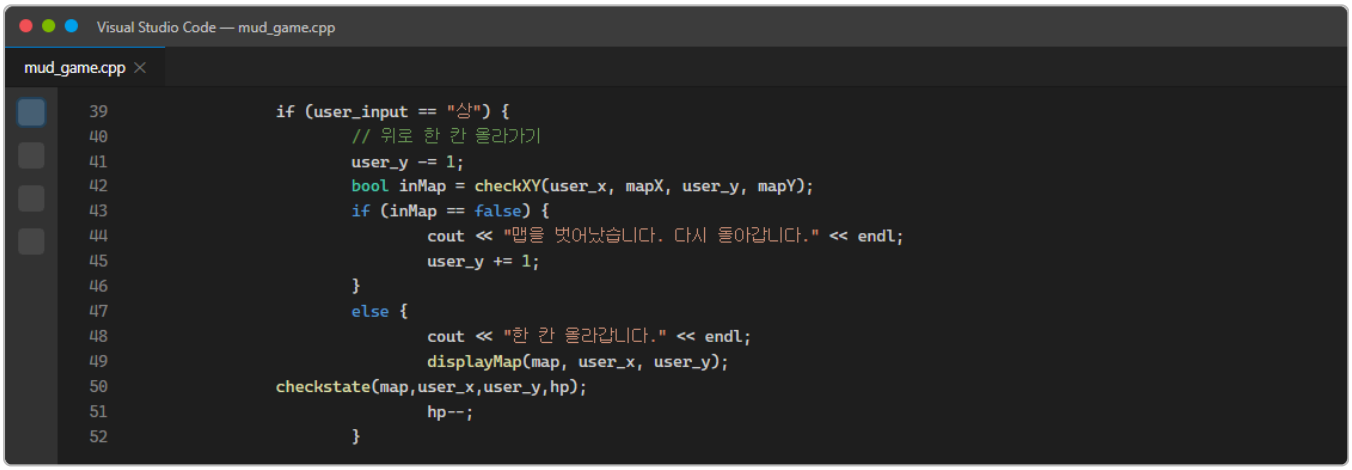


그림 3. '상' 명령 이동 처리 (하·좌·우도 동일한 구조)

입력	명령에 따른 변경 후 좌표, map, hp
반환값	없음 (메인 블록)
결과	이동 성공 시 지도 출력 + 칸 상태 처리 후 HP 1 감소 / 실패 시 원위치
설명	좌표를 바꾼 뒤 checkXY()로 경계를 검사하고, 지도 밖이면 메시지를 출력하고 방금 더한 값을 빼서 되돌린다. 통과하면 displayMap()으로 지도를 다시 그리고 checkState()로 그 칸의 사건을 처리한 뒤, `hp--`로 이동 비용을 적용한다. 이동 비용은 어떤 칸이든 무조건 발생하므로 checkState() 호출 밖에 두었다.

3.3 메인 함수 — 종료 조건 검사

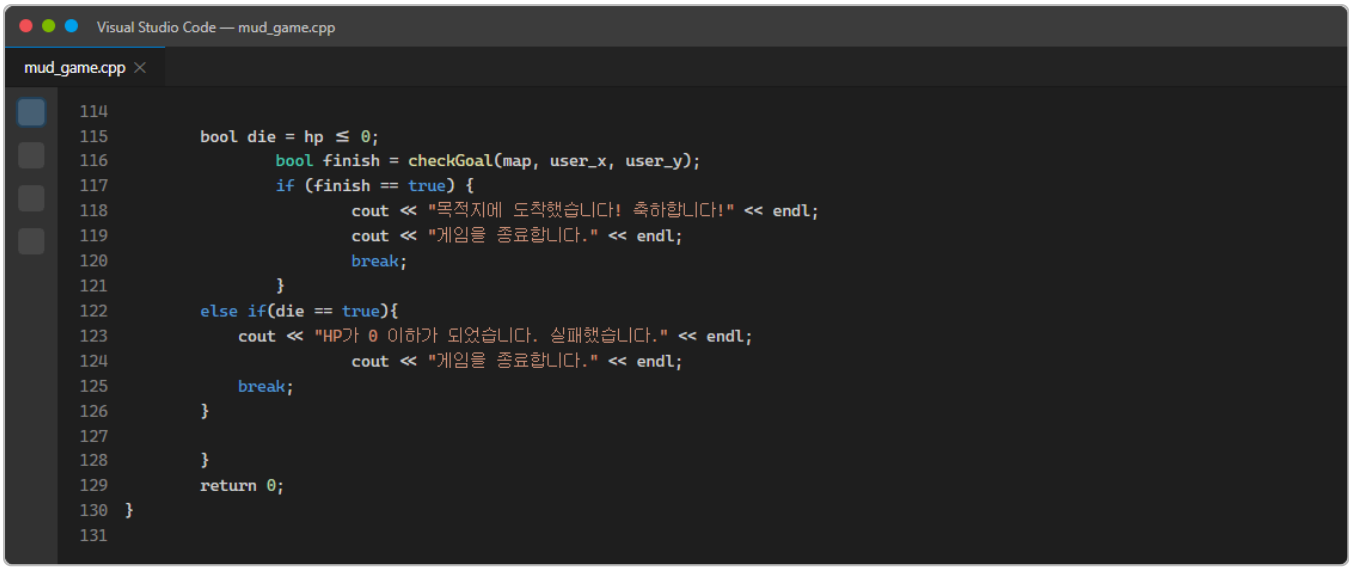


그림 4. 목적지 도착 / HP 실패 검사

이동이 끝날 때마다 checkGoal()으로 도착 여부를, `hp <= 0`으로 사멸 여부를 확인해 성공/실패 메시지를 출력하고 `break`로 종료한다.

3.4 displayMap() — 지도 출력 함수

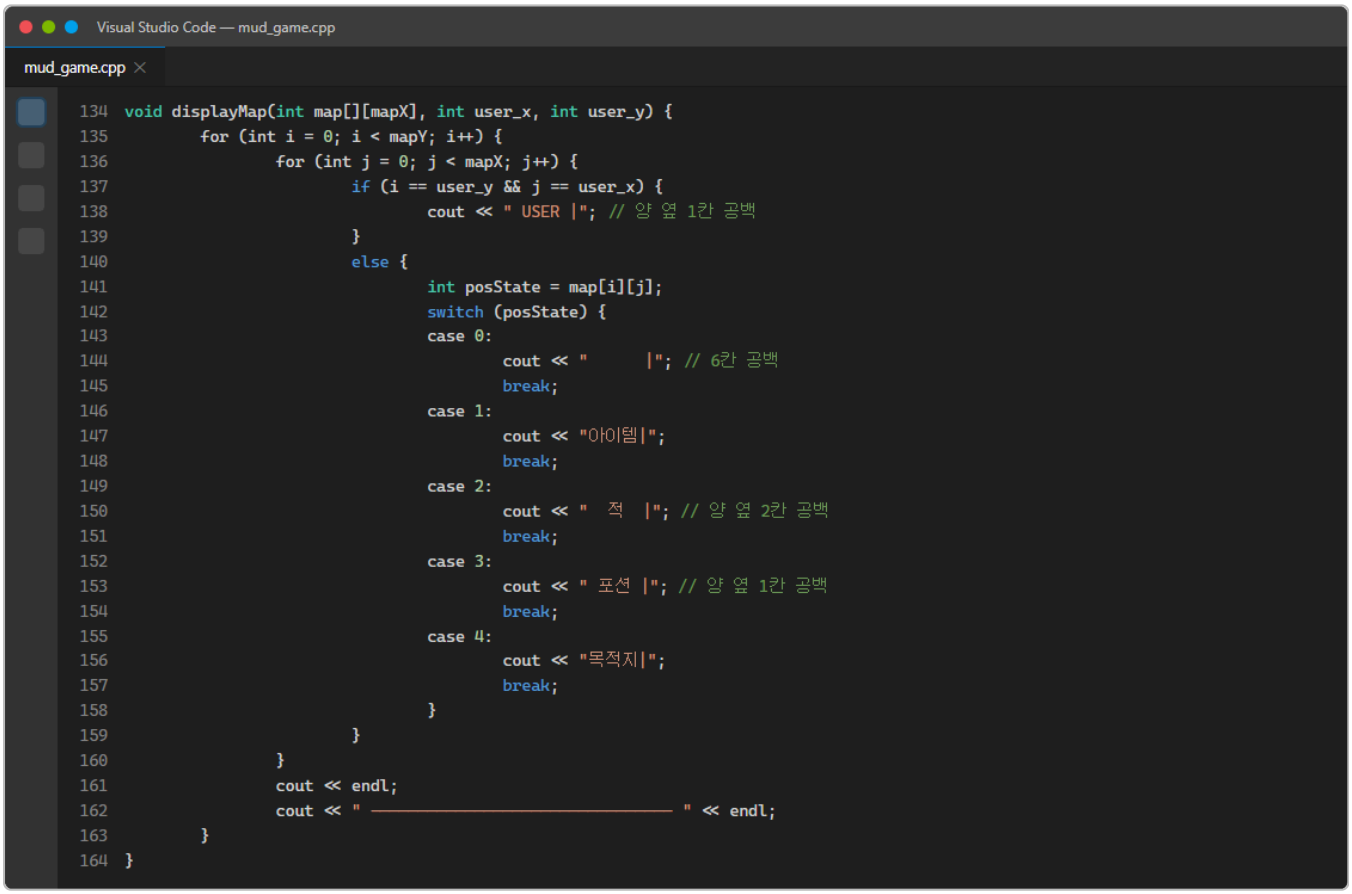


그림 5. displayMap() 구현

입력	int map[][] = 전체 지도 / user_x = 유저 x값 / user_y = 유저 y값
반환값	없음

결과	전체 지도 출력, 사용자 위치 출력
설명	2중 for문으로 2차원 배열 전체를 출력하다가 사용자 위치와 같은 좌표를 만나면 USER를 출력한다. 나머지 칸은 switch문으로 상태코드(0~4)에 따른 문구를 출력하며, 모든 칸을 폭 6칸으로 맞춰 격자가 흐트러지지 않게 했다.

3.5 checkXY() — 좌표 유효성 검사 함수

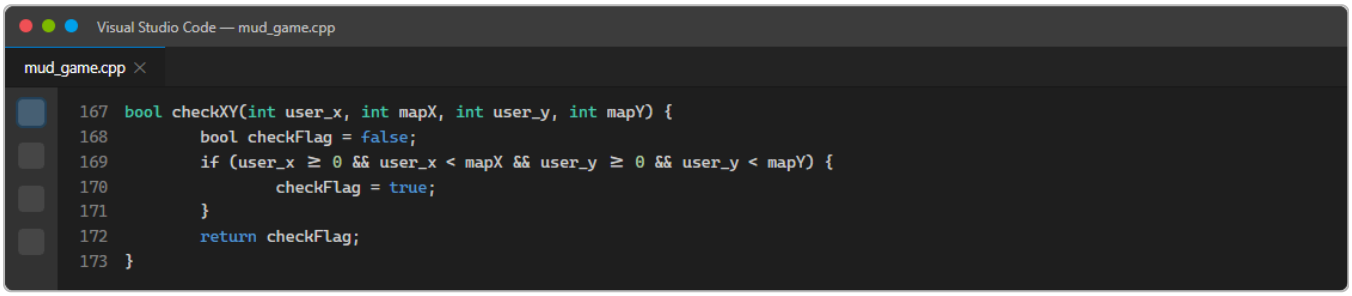


그림 6. checkXY() 구현

입력	user_x, user_y = 검사할 좌표 / mapX, mapY = 지도 크기
반환값	bool — 지도 내부면 true, 밖이면 false
결과	이동 좌표의 지도 내부 여부 판정
설명	x가 0 이상 mapX 미만이고 y가 0 이상 mapY 미만인지 AND(&&)로 검사한다. 음수 인덱스는 배열 접근 전에 걸러야 하므로 0 이상 조건이 반드시 필요하다.

3.6 checkGoal() — 목적지 도착 검사 함수

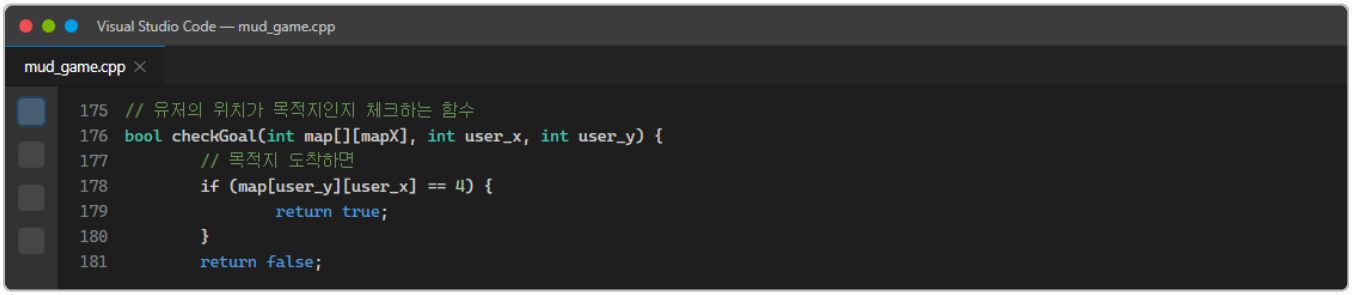


그림 7. checkGoal() 구현

입력	map = 지도 / user_x, user_y = 현재 좌표
반환값	bool — 목적지(4)면 true
결과	도착 여부 판정
설명	현재 좌표의 칸 값이 4인지 비교해 참/거짓을 반환한다.

3.7 checkState() — 칸 상태 처리 함수

Visual Studio Code — mud_game.cpp

mud_game.cpp

```cpp
//뭔갈 만나나?
void checkstate(int map[][mapX], int user_x, int user_y, int& hp){
    // 0은 빈 공간, 1은 아이템, 2는 적, 3은 포션, 4는 목적지

        if (map[user_y][user_x] == 1){
        cout << "아이템이 있습니다. hp가 1증가합니다!" << endl;
                //return false;    //기본체력깎이는지여부
                hp++;
        }
        else if(map[user_y][user_x] == 2){
                cout << "적이 있습니다. 체력이 2가 줄어듭니다!!" << endl;
                hp -= 2;
                //return false;
        }
        else if(map[user_y][user_x] == 3){
                cout << "포션이 있습니다!!!! HP가 2가 증가됩니다!" << endl;
                hp += 2;
                //return false;
        }
        else if(map[user_y][user_x] == 4){
                //return false;
        }
        else if(map[user_y][user_x] == 0){
                cout << "빈칸입니다~~ 빈칸은 늘 HP가 1감소합니다!" << endl;
                //return true;
                hp--;
        }
        /* else{
                //return false;
        }
 */
}
```

그림 8. checkState() 구현

입력	map = 지도 / user_x, user_y = 도착 좌표 / int& hp = 유저 체력(참조 전달)
반환값	없음 (void)
결과	칸 종류별 메시지 출력, 적 -2 / 포션 +2 / 아이템 +1 HP 반영
설명	도착한 칸의 상태코드를 if~else if 체인으로 비교한다. hp를 참조(&)로 받아 함수 안에서 직접 더하거나 빼면 호출자의 실제 체력이 갱신된다. 이동 시 감소는 모든 칸에 공통되는 규칙이라 호출 쪽에서 처리하고, 이 함수는 칸 고유의 사건만 담당하도록 void로 설계했다. 물건은 사라지지 않는다는 전제에 따라 map 배열은 수정하지 않는다.

4. 테스트

4.1 기능별 테스트 결과

```
C:\Users\wlstj\source\repos\mud_game\x64\Debug\mud_game.exe
현재 HP: 20  명령어를 입력하세요 (상,하,좌,우,지도,종료): 맵을 벗어났습니다. 다시 돌아갑니다.
```

그림 9. (지도 밖) 시작 위치에서 '상' → 에러 메시지 후 원위치

```
C:\Users\wlstj\source\repos\mud_game\x64\Debug\mud_game.exe
현재 HP: 20  명령어를 입력하세요 (상,하,좌,우,지도,종료): 잘못된 입력입니다.
```

그림 10. (잘못된 입력) 규칙 외 명령 → 에러 메시지 후 재입력



그림 11. (지도) '지도' 입력 → 전체 지도와 현재 위치(USER) 출력

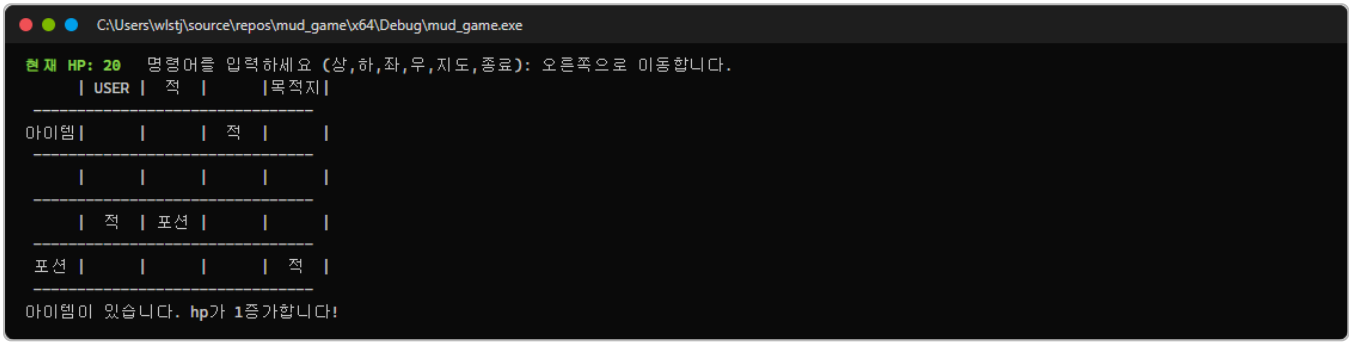


그림 12. (아이템) 아이템 칸 → hp +1 (이동 비용 -1과 상쇄되어 유지)

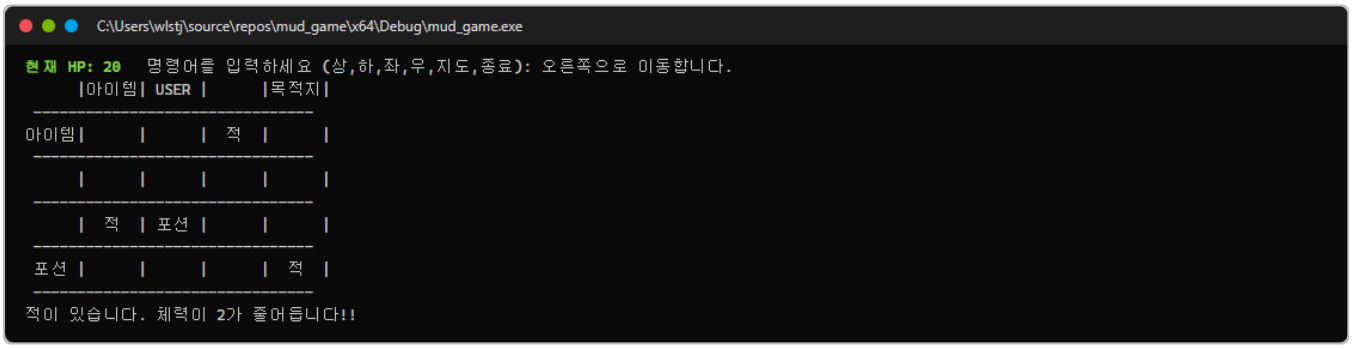


그림 13. (적) 적 칸 → HP 2 감소에 이동 비용 1까지 감소

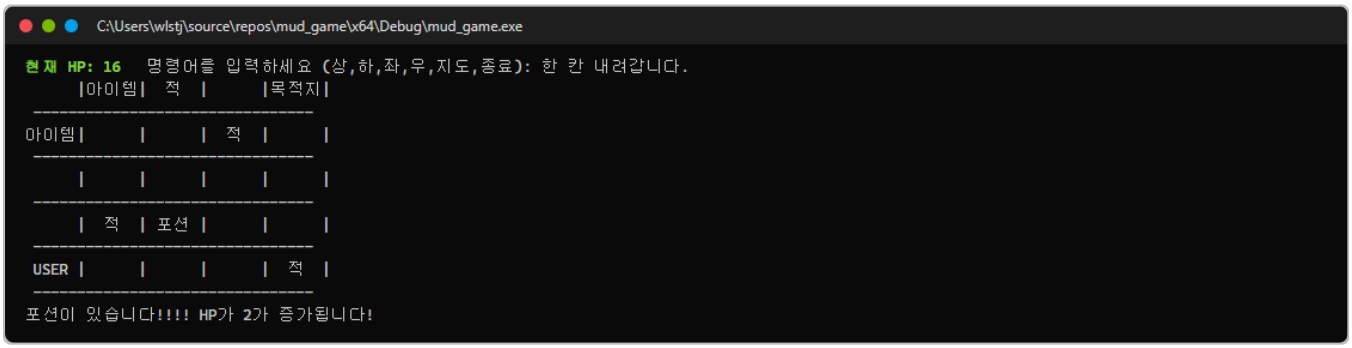


그림 14. (포션) 포션 칸 → HP 2 증가 메시지

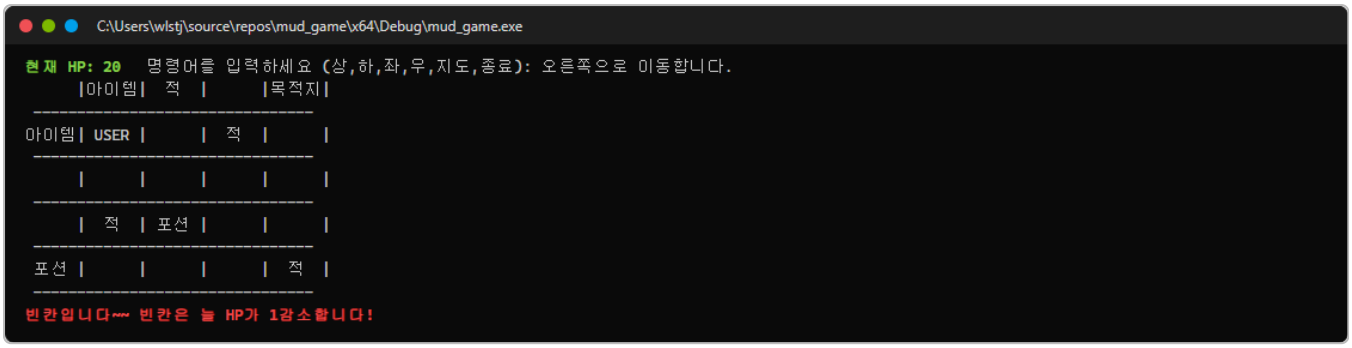


그림 15. (빈칸) 빈칸 칸 → 메시지 출력, 빈칸 감소 + 이동 비용

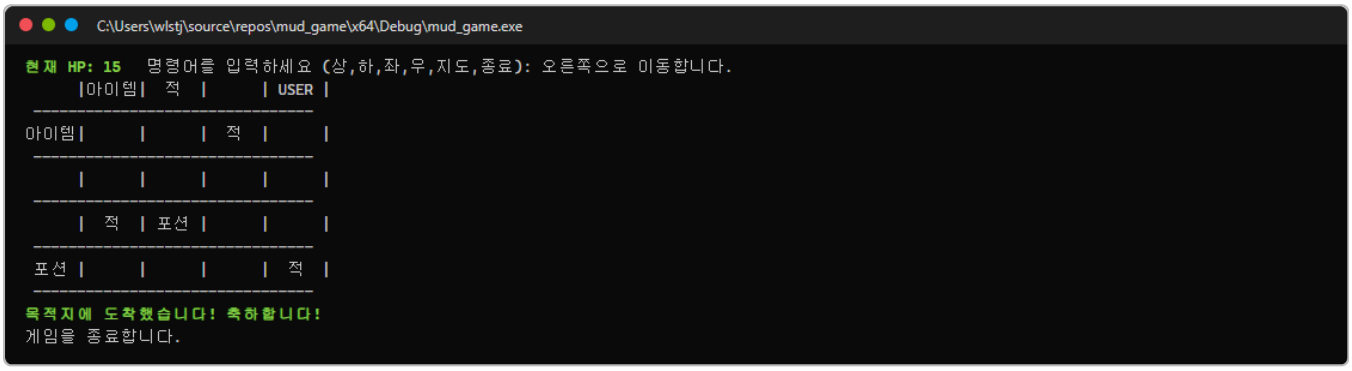


그림 16. (목적지) 목적지 도착 → 성공 메시지 후 종료

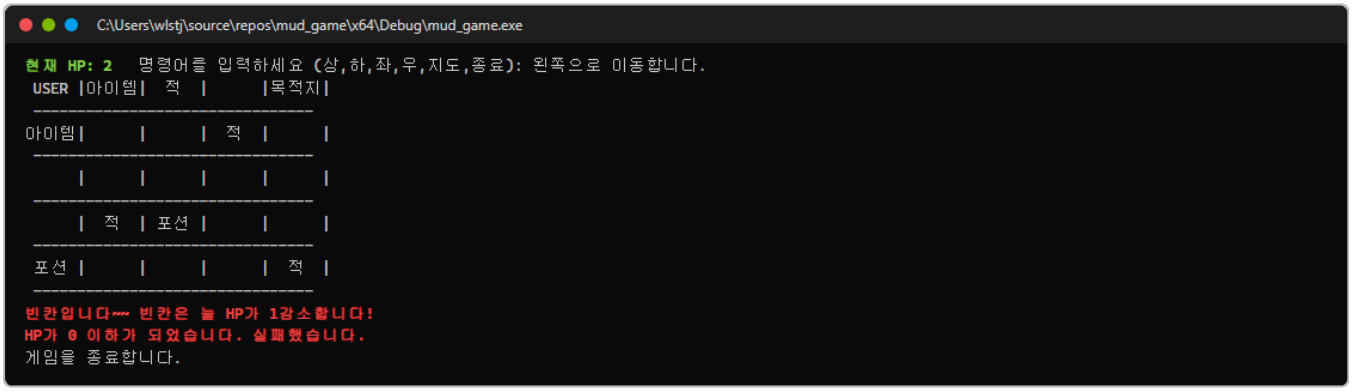


그림 17. (HP 실패) 좌·우 반복으로 HP 고갈 → 실패 메시지 후 종료

4.2 개발 중 문제를 고치면서 배운 점

1) HP 감소 대상을 '빈칸'에서 '이동'으로 바꾼 이유

처음에는 HP 감소를 빈칸 처리 안에만 넣었다. 빈칸에 서야만 체력이 깎이고, 적이나 포션 칸에서는 칸 효과만 적용되는 설계였다. 그런데 요구사항을 다시 읽어보니 "**이동할 때마다** 1씩 감소"는 칸과 무관한 기본 규칙이었다. 그래서 그림 3처럼 checkState() 호출 뒤에 `hp--`를 두어 어떤 칸이든 이동 자체 비용 1이 반드시 들게 고쳤고, 칸 고유의 증감은 checkState()가 그대로 담당하도록 역할을 나눴다. 같은 'HP 감소'라도 조건이 붙는 감소와 항상 발생하는 감소가 코드에서 위치가 달라진다는 점을 이번 수정으로 확실히 구분하게 되었다.

2) if 조건식의 함수 호출도 '실행'이다

수정 전에는 아래처럼 if 조건과 블록 안에서 checkState()를 한 번씩 불렀다.

```
if (checkState(map, user_x, user_y, hp)) {   // 이 순간 이미 함수 실행 (출력 포함)
    checkState(map, user_x, user_y, hp);     // 조건이 true일 때 또 실행
    hp--;
}
```

그러면 빈칸으로 이동했을 때 "빈칸입니다" 메시지가 **두 번** 출력됐다. 조건부에 쓴 함수 호출은 검사를 '해주는' 것이 아니라 그 자리에서 실제로 '실행'되어 출력까지 끝나 있고, 반환값이 true라서 블록에 들어가 두 번째 호출까지 실행된 것이다. 적이나 포션은 false를 반환해 블록을 타지 않으므로 한 번만 출력된 것도 같은 원리다. 이후 HP 차감 위치를 옮기면서 반환값이 필요 없어졌고, 함수를 void로 바꾸고 호출도 한 번으로 정리해 이 문제를 없앴다. 앞으로는 조건부 안에 출력이나 값 변경을 동반하는 함수를 넣을 때 "두 번 실행되면 어떻게 되는가"를 먼저 확인하려고 한다.

5. 결과 및 결론

5.1 프로젝트 결과

2차원 배열 기반 MUD 게임에 체력 시스템과 칸 상태 처리를 추가해 완성했다. 상·하·좌·우 이동, 경계 검사, 지도 출력, 잘못된 입력 처리, 아이템/적/포션 조우, 이동 시 HP 감소, 성공/실패 종료까지 요구사항 전체가 정상 동작함을 테스트로 확인했다.

5.2 느낀 점

요구사항 문장을 대충 읽고 '빈칸이면 깎이는 것'으로 해석했다가, 실행 화면을 보면서 이동 자체에도 감소가 붙어야 한다는 것을 뒤늦게 캐치한 것이 이번 작업에서 가장 큰 수확이었다. 결과 화면만 믿지 말고 요구사항을 다시 들여다보는 습관이 필요하다고 느꼈다. 또 조건부 안에서 함수가 실행된다는 사실을 버그를 통해 체득한 덕분에, 함수를 설계할 때 반환값·부작용·호출 횟수를 함께 따져야 한다는 것도 배웠다.